

다중 맵 — 자료구조 설계

날짜: 2026-05-03 선행: 그리드 리뉴얼 (마스터 69d21af) 원칙: 자료구조-행동 일치 — 의미 분리된 행동은 자료구조 결로도 분리

§0. 배경

0-1. 현 결 한계

현재 게임은 단일 16×12 그리드 1 장. `WORLD_TERRAIN` / `TILE_ENCOUNTERS` / `SPAWN_POS` 가 단수 결로 박힘. 다른 영역 (폐허 / 깊은 동굴 / 거점 마을 등) 으로 확장하려면 **공간 분리** 가 자료구조 결로 표현되어야 함.

0-2. 핵심 결정

- 위치 = 두 정보의 이원화: (`current_map_id`, `Vector2i`)
- 맵 단위 dict — 각 맵이 독립 자료 객체
- 슬롯 신규 kind: `transition` — 한 맵 → 다른 맵
- 진행 상태 (`visited` / `explored`) 는 맵 단위 dict
- 회차 통과 자원 (`big_run_data` — 카드 / 팔 / 연구) 은 맵 무관

0-3. 거부된 결 (간단)

(B) (`map_id`, `slot_id`) 복합 키 — 자동 namespace. 단 `depends_on` 결도 복합 → 코드 복잡 ↑.

(C) 단일 큰 그리드 — 16×12 → 60×40+ 결로 키워서 자연 wall 로 영역 분리. 자료구조 변경 0. 단 한 화면 표시 결 어려움 + UI 결 큰 변경 + 영역 의미 구조 X.

(D) 월드맵 그래프 + 로컬 그리드 — 이중 결. 우리 결과 안 맞음 (자료구조 두 결 = 행동 결도 두 결).

거부 이유 (자료구조-행동 일치): 맵 = 의미적으로 분리된 공간. 행동 = "맵 안 이동" / "맵 간 전이" 두 결. 자료구조도 두 결로 분리해야. (B)(D) 는 분리하되 추가 복잡도. (C) 는 분리 자체를 안 함.

→ (A) MAPS dict + 맵 단위 객체 가 행동-구조 일치 결.

§1. 자료구조 변경

1-1. GameData (game_data.gd)

제거:

```
const WORLD_TERRAIN = [...]
const TERRAIN_RULES = {...}      # 유지 (지형 코드는 전역)
const TILE_ENCOUNTERS = {...}
const SPAWN_POS = Vector2i(...)
```

신설:

```
const MAPS = {
  "field_01": {
    "name": "평원",
    "terrain": [
      "FFFFFFFFFFFFFFFF",
      "FGGGGGGGGGGGGGGF",
      ...
    ],
    "spawn": Vector2i(1, 6),
    "encounters": {
      Vector2i(3, 6): { "on_enter": {...} },
      Vector2i(7, 6): { "on_enter": {"id": "research_node_main", "kind": "rese"},
      ...
    },
  },
  "ruin_01": {
    "name": "폐허",
    "terrain": [...],
    "spawn": Vector2i(0, 5),      # 전이 진입점
    "encounters": {...},
  },
  "deep_01": {
    "name": "심부",
    "terrain": [...],
    "spawn": Vector2i(7, 0),
    "encounters": {...},
  },
}

const STARTING_MAP: String = "field_01"
```

`TERRAIN_RULES` 는 전역 그대로 (지형 코드 G/F/P/W/C 의 의미는 맵 무관).

1-2. RunManager.run_data

제거:

```
"player_pos": Vector2i,
"visited_tiles": Dict[Vector2i, bool],
```

```
"explored_tiles": Dict[Vector2i, bool],
```

신설:

```
"current_map_id": String,                # 현 맵
"player_pos": Vector2i,                  # 그 맵 안 좌표
"visited_by_map": Dict[String, Dict[Vector2i, bool]],
"explored_by_map": Dict[String, Dict[Vector2i, bool]],
```

유지:

```
"seen_this_run": Dict[String, bool],    # 슬롯 id 글로벌 unique 컨벤션
"day", "actions_remaining", "actions_per_day", "phase", ...
"pending_combat": {...},
```

`seen_this_run` 은 글로벌 — 슬롯 id 가 맵 경로 unique 라는 컨벤션 (예: `ruin_signal_chain_2`).

1-3. 슬롯 — 신규 kind `transition`

```
{
  "id": "field_to_ruin",
  "kind": "transition",
  "target_map": "ruin_01",
  "target_pos": Vector2i(0, 5),
  # once_per: 미설정 (매번 자유 이동)
}
```

위치 결 — 보통 맵의 가장자리 (예: 평원 (15, 6) 동쪽 끝). 양방향 결로 박으려면 양쪽에 대응 슬롯.

```
# field_01 의 (15, 6) — 폐허로 진입
Vector2i(15, 6): {
  "on_enter": {
    "id": "field_to_ruin",
    "kind": "transition",
    "target_map": "ruin_01",
    "target_pos": Vector2i(0, 5),
  }
}

# ruin_01 의 (0, 5) — 평원으로 복귀
Vector2i(0, 5): {
  "on_enter": {
    "id": "ruin_to_field",
    "kind": "transition",
    "target_map": "field_01",
    "target_pos": Vector2i(15, 6),
  }
}
```

```
}  
}
```

§2. 매니저 단 변경

2-1. `_start_internal_run` (RunManager)

```
func _start_internal_run() -> void:  
    run_data = big_run_data.duplicate(true)  
    run_data["current_map_id"] = GameData.STARTING_MAP  
    var spawn: Vector2i = GameData.MAPS[GameData.STARTING_MAP]["spawn"]  
    run_data["player_pos"] = spawn  
    run_data["day"] = 1  
    run_data["day_max"] = GameData.DAY_MAX  
    run_data["actions_per_day"] = GameData.ACTIONS_PER_DAY  
    run_data["actions_remaining"] = GameData.ACTIONS_PER_DAY  
    run_data["visited_by_map"] = { GameData.STARTING_MAP: { spawn: true } }  
    run_data["explored_by_map"] = { GameData.STARTING_MAP: {} }  
    run_data["seen_this_run"] = {}  
    run_data["pending_combat"] = {}  
    run_data["phase"] = "map"  
    state_changed.emit()  
    var run_start_event_id: String = EventManager.resolve_event("run_start", {})  
    if run_start_event_id != "":  
        _begin_event_phase(run_start_event_id)
```

회귀 시 모든 맵의 visited / explored 자동 빔 (= `_start_internal_run` 이 새로 박음).

2-2. 지형 / 슬롯 조회자

```
func _current_map() -> Dictionary:  
    return GameData.MAPS.get(run_data.get("current_map_id", ""), {})  
  
func _get_terrain_at(pos: Vector2i) -> String:  
    var map: Dictionary = _current_map()  
    var rows: Array = map.get("terrain", [])  
    if pos.y < 0 or pos.y >= rows.size():  
        return ""  
    var row: String = rows[pos.y]  
    if pos.x < 0 or pos.x >= row.length():  
        return ""  
    return row[pos.x]  
  
func get_tile_encounter(pos: Vector2i) -> Dictionary:  
    return _current_map().get("encounters", {}).get(pos, {})
```

`_is_passable` / `get_terrain_name` 은 변경 없음 (`_get_terrain_at` 위에 얹혀있음).

2-3. `try_move`

```
func try_move(dir: Vector2i) -> bool:
    if run_data.get("phase") != "map":
        return false
    if run_data.get("actions_remaining", 0) < 1:
        return false
    var target: Vector2i = run_data["player_pos"] + dir
    if not _is_passable(target):
        return false

    run_data["player_pos"] = target
    run_data["actions_remaining"] -= 1
    var map_id: String = run_data["current_map_id"]
    run_data["visited_by_map"][map_id][target] = true

    _check_on_enter(target)

    if run_data.get("phase") == "map" and run_data["actions_remaining"] <= 0:
        rest()

    state_changed.emit()
    return true
```

방문 추적 결만 변경 — `visited_by_map[current_map_id]` 결로.

2-4. `_check_on_enter` 의 신규 분기

```
match kind:
    "event": ...
    "combat": ...
    "research": ...
    "transition":
        run_data["seen_this_run"][slot.get("id", "")] = true
        var target_map: String = slot.get("target_map", "")
        var target_pos: Vector2i = slot.get("target_pos", Vector2i.ZERO)
        if not GameData.MAPS.has(target_map):
            push_warning("transition: 알 수 없는 target_map '%s'" % target_map)
            return
        run_data["current_map_id"] = target_map
        run_data["player_pos"] = target_pos
        if not run_data["visited_by_map"].has(target_map):
            run_data["visited_by_map"][target_map] = {}
        if not run_data["explored_by_map"].has(target_map):
            run_data["explored_by_map"][target_map] = {}
        run_data["visited_by_map"][target_map][target_pos] = true
```

```
# 도착지 슬롯 평가 (= 도착하자마자 진입 트리거 발화 가능)
_check_on_enter(target_pos)
```

전이 자체는 행동력 1 만 소모 (이동과 동일). 도착지에서 이벤트 / 전투 발화는 그대로 일어남.

2-5. try_explore

```
func try_explore() -> bool:
    if run_data.get("phase") != "map":
        return false
    if run_data.get("actions_remaining", 0) < 1:
        return false
    var pos: Vector2i = run_data["player_pos"]
    var map_id: String = run_data["current_map_id"]
    if run_data["explored_by_map"][map_id].get(pos, false):
        return false

    run_data["explored_by_map"][map_id][pos] = true
    run_data["actions_remaining"] -= 1
    _check_on_explore(pos)

    if run_data.get("phase") == "map" and run_data["actions_remaining"] <= 0:
        rest()

    state_changed.emit()
    return true
```

§3. UI 단 변경

3-1. _build_grid_display

현 결: WORLD_TERRAIN 위에 1회 빌드.

신 결: current_map_id 변경 시 다시 빌드. 또는 모든 맵의 그리드를 미리 만들어두고 보이기/숨기기.

옵션 (a) — Lazy 다시 빌드:

```
var _current_built_map: String = ""

func _refresh_grid_display() -> void:
    var map_id: String = RunManager.run_data.get("current_map_id", "")
    if map_id != _current_built_map:
        _rebuild_grid(map_id)
        _current_built_map = map_id
    # ... 마커 / 플레이어 갱신
```

옵션 (b) — 미리 빌드 + 토글:

```
var _grid_roots: Dictionary = {} # map_id → Control

func _build_grid_display() -> void:
    for map_id in GameData.MAPS.keys():
        _grid_roots[map_id] = _build_one_grid(map_id)
        _grid_roots[map_id].visible = false
        add_child(_grid_roots[map_id])

func _refresh_grid_display() -> void:
    var current: String = RunManager.run_data.get("current_map_id", "")
    for map_id in _grid_roots.keys():
        _grid_roots[map_id].visible = (map_id == current)
    # ... 마커 / 플레이어 갱신
```

(b) 가 메모리 ↑ 단순함 ↑. 맵 수 < 10 이면 (b) 채택.

3-2. HUD — 현 맵 이름 표시

```
_terrain_label.text = "%s - %s (%d, %d)" % [
    GameData.MAPS[map_id]["name"],
    RunManager.get_terrain_name(pos),
    pos.x, pos.y,
]
```

§4. 세이브 / 로드

4-1. 직렬화

`run_data["current_map_id"]` (String) — 단순 직렬화. `run_data["visited_by_map"]` — String → Dict[Vector2i, bool] 의 중첩 dict. `var_to_str` 호환 OK (Vector2i 키 OK 이미 확인).

```
"visited_by_map": {
    "field_01": { Vector2i(1,6): true, Vector2i(3,6): true },
    "ruin_01": { Vector2i(0,5): true },
}
```

4-2. 로드 시 마이그레이션

기존 세이브 (`current_map_id` 필드 없음) 대응:

```
# game_manager.gd::load_save 직후
func _migrate_save(rd: Dictionary, brd: Dictionary) -> void:
```

```

if not rd.has("current_map_id"):
    rd["current_map_id"] = GameData.STARTING_MAP
if not rd.has("visited_by_map"):
    rd["visited_by_map"] = {
        GameData.STARTING_MAP: rd.get("visited_tiles", {}).duplicate()
    }
    rd.erase("visited_tiles")
if not rd.has("explored_by_map"):
    rd["explored_by_map"] = {
        GameData.STARTING_MAP: rd.get("explored_tiles", {}).duplicate()
    }
    rd.erase("explored_tiles")

```

§5. 회귀 (end_internal_run / _start_internal_run)

회귀 시 — 모든 맵 진행 자동 리셋. - `current_map_id` → `STARTING_MAP` - `player_pos` → `MAPS[STARTING_MAP]["spawn"]` - `visited_by_map` → 시작 맵의 spawn 만 박힘 - `explored_by_map` → 빈 - `seen_this_run` → 빈

회차 통과 자원 (`big_run_data.seen_events` / 카드 / 팔 / 연구) 은 맵 무관 그대로.

§6. 검증 요구사항

6-1. 헤드리스 신규 항목

- `MAPS` 의 모든 맵 키 / `terrain` 크기 / `spawn` 좌표 검증
- `STARTING_MAP` 이 `MAPS` 에 존재
- 모든 transition 슬롯의 `target_map` 이 `MAPS` 에 존재
- `target_pos` 가 해당 맵의 `terrain` 범위 내 + passable
- 회귀 시 `visited_by_map` / `explored_by_map` 비움 + `current_map_id` = `STARTING`

6-2. 시뮬 회귀

- 맵 A → 맵 B 전이 → `current_map_id` 변경 + `player_pos` 변경
- 맵 B 의 슬롯 발화
- 맵 B → 맵 A 복귀
- 회귀 → `STARTING_MAP` 으로 자동 복귀
- 세이브 시점이 맵 B → 로드 시 맵 B 그대로

6-3. UI 시각

- 현 맵 그리드 표시 + 다른 맵 그리드 숨김
 - 전이 슬롯 마커 (신규 색)
 - HUD 의 맵 이름 표시
-

§7. 단계별 구현 롤아웃

M-1 — 자료구조 + 시뮬 (기존 1 맵 유지하며 schema 만 다중 맵 결로)

- `MAPS` const 신설 (`field_01` 1 개로 시작 — 기존 결 그대로 wrapping)
- `WORLD_TERRAIN` / `TILE_ENCOUNTERS` / `SPAWN_POS` 폐기 (`MAPS["field_01"]` 안으로)
- `run_data` 의 신규 필드 (`current_map_id` 등)
- `_start_internal_run` / `try_move` / `try_explore` / `_check_on_enter` 갱신
- 헤드리스 검증: 1 맵 결의 모든 기존 동작 유지

→ 회귀 186 PASS 유지 가 검증 기준.

M-2 — 두번째 맵 + transition 슬롯

- `MAPS` 에 `ruin_01` 추가 (10×8 또는 비슷 작은 결, 슬롯 5~8 개)
- `field_01` 의 (15, 6) 에 transition → `ruin_01`
- `ruin_01` 의 (0, 5) 에 transition → `field_01`
- `_check_on_enter` 의 transition 분기 구현
- 시뮬 검증: 전이 / 복귀 / 회귀

M-3 — UI 다중 그리드

- `_build_grid_display` 가 맵별 그리드 미리 빌드 (옵션 b)
- `_refresh_grid_display` 가 `current_map_id` 결로 가시성 토글
- HUD 의 맵 이름 표시
- 시각 검증 1~2 컷 (맵 A / 맵 B)

M-4 — 세이브 마이그레이션

- `_migrate_save` 결로 기존 세이브 호환
- 세이브 직렬화 검증 (`current_map_id` / `visited_by_map`)

M-5 — 추가 맵 (선택)

- `deep_01` / 거점 맵 등 콘텐츠 차로 추가. 시스템 변경 없음.

합 시간: M-1 ~ M-4 약 4~5 시간.

§8. 외부 동작 변경

- M-1 — 외부 동작 변경 0 (1 맵 결의 모든 행동 동일)
 - M-2 — 신규 행동 추가 (전이). 기존 행동 영향 0.
 - M-3 — UI 결만 변경. 시물 영향 0.
 - M-4 — 세이브 호환 결만 변경. 새 세이브 동작 동일.
-

§9. 비교

9-1. 슬롯 id namespace

글로벌 unique 컨벤션. 작성 시 맵 prefix 박는 걸 권장: - `ruin_signal_chain_2` (= ruin_01의 `signal_chain_2`) - `field_regression_speech` 또는 그냥 `regression_speech` (시작 맵은 prefix 생략 OK)

9-2. depends_on 결

`depends_on: ["ruin_signal_chain_2"]` — 슬롯 id 가 글로벌 unique 라 그대로 작동. 다른 맵의 슬롯 결도 의존 가능 (예: 폐허에서 본 것을 평원에서 활용).

9-3. 한 맵 안 다른 입구 결

한 맵에 여러 진입점 — 각 transition 의 `target_pos` 가 다르면 됨. `MAPS[map_id]["spawn"]` 은 회귀 시 시작 맵 진입 결로만 쓰임.

9-4. 적/이벤트 풀의 맵별 결

`ENEMIES` / `EVENTS` 는 전역 그대로. 각 맵의 슬롯이 어느 적 / 이벤트를 쓰는지 결만 정함. 같은 적이 여러 맵에 등장 가능.

9-5. 카메라 / 큰 맵 결

한 맵 = 16×12 결로 한 화면 표시 가능 결. 맵 크기 결이 더 커지면 (예: 32×24) 카메라 / 스크롤 / 미니맵 신규 결 필요. 본 차로는 16×12 단일 결 유지.

§10. 다음 액션

본 spec 합의 후: 1. **M-1** — 자료구조 + 시물 (기존 1 맵 유지). 회귀 186 PASS 검증. 2. **M-2** — 두번째 맵 + transition. 3. **M-3** — UI 다중 그리드. 4. **M-4** — 세이브 마이그레이션.

기존 콘텐츠 (`regression_speech` / `repair_choice` / `found_letter` / `research_node_main` / `larva_combat_4_4`) 는 모두 `field_01` 안에 남음. 동작 그대로.

§11. 미해결 결

- (a) 첫 게임 진입 시 시작 맵 = `field_01` 결 OK?
- (b) UI 결 — 그리드 미리 빌드 (옵션 b) vs lazy (옵션 a)?
- (c) 슬롯 id prefix 컨벤션 — 시작 맵도 prefix 박을지?
- (d) 맵 간 회차 카운터 / 전이 횟수 추적 결 필요한지? (본 spec 에는 없음)

답 받은 후 M-1 진입.